EXPERIMENT 6: NAÏVE BAYES CLASSIFIER

Experiment Question: Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Naïve Bayes: General concept and mathematical rule

Naïve Bayes is a probabilistic classifier based on Bayes' theorem: $P(C \mid X) = P(C) \times P(X \mid C) / P(X)$. It is called naïve because it assumes that the features are conditionally independent given the class, so $P(X \mid C) = P(x_1 \mid C) \times P(x_2 \mid C) \times \ldots \times P(x_n \mid C)$.

Prediction rule: since $P(X)$ is the same for all classes, the predicted class is the one with the largest value of $P(C) \times \Pi\, P(x_i \mid C)$. For categorical attributes $P(x_i \mid C)$ is a relative frequency (with Laplace smoothing to avoid zero); for numeric attributes a Gaussian density is used; for text, word counts are used and log probabilities are added.

Confusion matrix: a table whose rows are the actual classes and whose columns are the predicted classes. For two classes, TP and TN are correct predictions, while FP and FN are errors. Accuracy = (TP + TN) / (TP + TN + FP + FN).

Important: the accuracy is measured on the small test sets listed in each subproblem, so it is only an estimate; Subproblem 4 loads the Iris dataset from scikit-learn (pip install scikit-learn).

Subproblem 1: Play Tennis Prediction (Categorical Attributes)

1. Aim

To implement the Naïve Bayes classifier for categorical data and evaluate it on test samples using a confusion matrix and accuracy.

2. Concept

The attributes Outlook, Temperature, Humidity, and Wind are categorical. Laplace smoothing ($\alpha = 1$) avoids zero probabilities. The model is trained on 14 days and tested on 7 new days.

3. Dataset Description

Training data (14 rows):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Test data (7 rows):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	Normal	Weak	Yes
Overcast	Cool	High	Strong	Yes
Rain	Mild	High	Strong	No
Sunny	Mild	High	Weak	No
Rain	Cool	Normal	Weak	Yes
Sunny	Cool	Normal	Strong	Yes
Sunny	Cool	High	Strong	Yes

4. Algorithm

1. Separate the data into training and test sets.
2. Compute the prior probability P(class) of every class from the training data.

3. For every attribute value, compute the conditional probability P(value | class) with Laplace smoothing.
4. For a test sample, multiply the prior by the conditional probabilities of its attribute values for each class.
5. Predict the class with the largest product.
6. Build the confusion matrix from actual and predicted classes and compute the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Priors: P(No) = 5/14 = 0.3571; P(Yes) = 9/14 = 0.6429.
2. Take test sample 1: ('Sunny', 'Hot', 'Normal', 'Weak') (actual class Yes). Conditional probabilities with Laplace smoothing, P(value | class) = (count + 1) / (N_class + V):
3. P(Outlook = Sunny | No) = (3 + 1)/(5 + 3) = 0.5000; P(Temperature = Hot | No) = (2 + 1)/(5 + 3) = 0.3750; P(Humidity = Normal | No) = (1 + 1)/(5 + 2) = 0.2857; P(Wind = Weak | No) = (2 + 1)/(5 + 2) = 0.4286.
4. P(Outlook = Sunny | Yes) = (2 + 1)/(9 + 3) = 0.2500; P(Temperature = Hot | Yes) = (2 + 1)/(9 + 3) = 0.2500; P(Humidity = Normal | Yes) = (6 + 1)/(9 + 2) = 0.6364; P(Wind = Weak | Yes) = (6 + 1)/(9 + 2) = 0.6364.
5. Score(No) = P(No) × Π $P(x_i \mid No)$ = 0.3571 × 0.5000 × 0.3750 × 0.2857 × 0.4286 = 0.008200.
6. Score(Yes) = P(Yes) × Π $P(x_i \mid Yes)$ = 0.6429 × 0.2500 × 0.2500 × 0.6364 × 0.6364 = 0.016271.
7. The larger score belongs to Yes, so test sample 1 is predicted as Yes. The same calculation is repeated for every test sample.
8. Confusion matrix (rows = actual, columns = predicted): actual No: [2, 0]; actual Yes: [1, 4], with columns in the order No, Yes.
9. Accuracy = (2 + 4) / 7 = 6/7 = 85.71%.
10. Misclassified test sample(s): sample 7 (actual Yes, predicted No).

6. Formulae

- Bayes' theorem: $P(C \mid X) = P(C) \cdot P(X \mid C) / P(X)$
- Naïve assumption: $P(X \mid C) = P(x_1 \mid C) \cdot P(x_2 \mid C) \cdot \ldots \cdot P(x_n \mid C)$
- Decision rule: predicted class = argmax over C of $P(C) \cdot \Pi P(x_i \mid C)$
- Laplace smoothing: $P(x_i = v \mid C) = (count(v, C) + \alpha) / (N_C + \alpha \cdot V)$, with $\alpha = 1$ and V = number of values of the attribute
- Confusion matrix: rows = actual class, columns = predicted class; the diagonal holds the correct predictions
- Accuracy = (TP + TN) / (TP + TN + FP + FN) = (sum of the diagonal) / (total test samples)

7. Python Program

```
from collections import Counter

def train_nb(X, y, alpha=1):
    classes = sorted(set(y))
    priors = {c: y.count(c) / len(y) for c in classes}
    values = [sorted(set(row[j] for row in X)) for j in range(len(X[0]))]
    cond = {}
```

```
    for c in classes:
        rows_c = [x for x, label in zip(X, y) if label == c]
        for j in range(len(X[0])):
            for v in values[j]:
                count = sum(1 for r in rows_c if r[j] == v)
                cond[(c, j, v)] = (count + alpha) / (len(rows_c) + alpha *
len(values[j]))
    return classes, priors, cond

def predict(x, model):
    classes, priors, cond = model
    scores = {}
    for c in classes:
        p = priors[c]
        for j, v in enumerate(x):
            p *= cond.get((c, j, v), 1e-9)
        scores[c] = p
    return max(scores, key=scores.get), scores

def confusion_matrix(actual, predicted, labels):
    matrix = [[0] * len(labels) for _ in labels]
    for a, p in zip(actual, predicted):
        matrix[labels.index(a)][labels.index(p)] += 1
    return matrix

def show_results(actual, predicted, labels):
    matrix = confusion_matrix(actual, predicted, labels)
    w = max(len(l) for l in labels) + 8
    print("\nConfusion Matrix (rows = actual, columns = predicted)")
    print(" " * (w + 2) + "".join(("Pred " + l).rjust(w) for l in labels))
    for label, row in zip(labels, matrix):
        print(("Actual " + label).ljust(w + 2) + "".join(str(v).rjust(w) for v in
row))
    correct = sum(matrix[i][i] for i in range(len(labels)))
    print("\nAccuracy = %d / %d = %.2f%%" % (correct, len(actual), correct /
len(actual) * 100))

# Training data: Outlook, Temperature, Humidity, Wind -> PlayTennis
train = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'], ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'], ['Rain', 'Mild', 'High', 'Weak',
'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'], ['Rain', 'Cool', 'Normal', 'Strong',
'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'], ['Sunny', 'Mild', 'High', 'Weak',
'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'], ['Rain', 'Mild', 'Normal', 'Weak',
'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'], ['Overcast', 'Mild', 'High',
'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'], ['Rain', 'Mild', 'High', 'Strong',
'No']]
test = [
    ['Sunny', 'Hot', 'Normal', 'Weak', 'Yes'], ['Overcast', 'Cool', 'High', 'Strong',
'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No'], ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'], ['Sunny', 'Cool', 'Normal', 'Strong',
'Yes'], ['Sunny', 'Cool', 'High', 'Strong', 'Yes']]

X_train, y_train = [r[:-1] for r in train], [r[-1] for r in train]
X_test, y_test = [r[:-1] for r in test], [r[-1] for r in test]

model = train_nb(X_train, y_train)
```

```
print("Prior probabilities:", {c: round(p, 4) for c, p in model[1].items()})

predicted = []
for x, actual in zip(X_test, y_test):
    label, scores = predict(x, model)
    predicted.append(label)
    print(x, " actual =", actual, " predicted =", label)

show_results(y_test, predicted, ["No", "Yes"])
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/6.1.py ====
Prior probabilities: {'No': 0.3571, 'Yes': 0.6429}
['Sunny', 'Hot', 'Normal', 'Weak']  actual = Yes  predicted = Yes
['Overcast', 'Cool', 'High', 'Strong']  actual = Yes  predicted = Yes
['Rain', 'Mild', 'High', 'Strong']  actual = No  predicted = No
['Sunny', 'Mild', 'High', 'Weak']  actual = No  predicted = No
['Rain', 'Cool', 'Normal', 'Weak']  actual = Yes  predicted = Yes
['Sunny', 'Cool', 'Normal', 'Strong']  actual = Yes  predicted = Yes
['Sunny', 'Cool', 'High', 'Strong']  actual = Yes  predicted = No

Confusion Matrix (rows = actual, columns = predicted)
                Pred No   Pred Yes
Actual No             2          0
Actual Yes            1          4

Accuracy = 6 / 7 = 85.71%
>>>
```

9. Result / Conclusion

The Naïve Bayes classifier was implemented successfully. The confusion matrix was displayed and the accuracy on the test data is 85.71% (1 test sample(s) misclassified).

Subproblem 2: Loan Approval Prediction

1. Aim

To use Naïve Bayes to predict whether a loan application is approved and to display the confusion matrix and accuracy.

2. Concept

The attributes are Income, Employment, and CreditHistory. The model is trained on 12 past applications and tested on 6 other applications.

3. Dataset Description

Training data (12 rows):

Income	Employment	CreditHistory	LoanApproved
High	Salaried	Good	Yes
High	SelfEmployed	Good	Yes
High	Salaried	Poor	No
Medium	Salaried	Good	Yes
Medium	SelfEmployed	Poor	No
Medium	SelfEmployed	Good	Yes
Low	Salaried	Good	Yes
Low	SelfEmployed	Good	No
Low	Salaried	Poor	No
Low	SelfEmployed	Poor	No
Medium	Salaried	Poor	No
High	SelfEmployed	Poor	No

Test data (6 rows):

Income	Employment	CreditHistory	LoanApproved
High	Salaried	Good	Yes
Low	SelfEmployed	Poor	No
Medium	Salaried	Poor	No
Low	Salaried	Good	Yes
High	SelfEmployed	Good	Yes
Medium	SelfEmployed	Poor	Yes

4. Algorithm

1. Separate the data into training and test sets.
2. Compute the prior probability P(class) of every class from the training data.
3. For every attribute value, compute the conditional probability P(value | class) with Laplace smoothing.
4. For a test sample, multiply the prior by the conditional probabilities of its attribute values for each class.

5. Predict the class with the largest product.
6. Build the confusion matrix from actual and predicted classes and compute the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Priors: P(No) = 7/12 = 0.5833; P(Yes) = 5/12 = 0.4167.
2. Take test sample 1: ('High', 'Salaried', 'Good') (actual class Yes). Conditional probabilities with Laplace smoothing, P(value | class) = (count + 1) / (N_class + V):
3. P(Income = High | No) = (2 + 1)/(7 + 3) = 0.3000; P(Employment = Salaried | No) = (3 + 1)/(7 + 2) = 0.4444; P(CreditHistory = Good | No) = (1 + 1)/(7 + 2) = 0.2222.
4. P(Income = High | Yes) = (2 + 1)/(5 + 3) = 0.3750; P(Employment = Salaried | Yes) = (3 + 1)/(5 + 2) = 0.5714; P(CreditHistory = Good | Yes) = (5 + 1)/(5 + 2) = 0.8571.
5. Score(No) = P(No) × Π P(x_i | No) = 0.5833 × 0.3000 × 0.4444 × 0.2222 = 0.017284.
6. Score(Yes) = P(Yes) × Π P(x_i | Yes) = 0.4167 × 0.3750 × 0.5714 × 0.8571 = 0.076531.
7. The larger score belongs to Yes, so test sample 1 is predicted as Yes. The same calculation is repeated for every test sample.
8. Confusion matrix (rows = actual, columns = predicted): actual No: [2, 0]; actual Yes: [1, 3], with columns in the order No, Yes.
9. Accuracy = (2 + 3) / 6 = 5/6 = 83.33%.
10. Misclassified test sample(s): sample 6 (actual Yes, predicted No).

6. Formulae

- Bayes' theorem: P(C | X) = P(C) · P(X | C) / P(X)
- Naïve assumption: P(X | C) = P(x_1 | C) · P(x_2 | C) · … · P(x_n | C)
- Decision rule: predicted class = argmax over C of P(C) · Π P(x_i | C)
- Laplace smoothing: P(x_i = v | C) = (count(v, C) + α) / (N_C + α · V), with α = 1 and V = number of values of the attribute
- Confusion matrix: rows = actual class, columns = predicted class; the diagonal holds the correct predictions
- Accuracy = (TP + TN) / (TP + TN + FP + FN) = (sum of the diagonal) / (total test samples)

7. Python Program

```
from collections import Counter

def train_nb(X, y, alpha=1):
    classes = sorted(set(y))
    priors = {c: y.count(c) / len(y) for c in classes}
    values = [sorted(set(row[j] for row in X)) for j in range(len(X[0]))]
    cond = {}
    for c in classes:
        rows_c = [x for x, label in zip(X, y) if label == c]
        for j in range(len(X[0])):
            for v in values[j]:
                count = sum(1 for r in rows_c if r[j] == v)
                cond[(c, j, v)] = (count + alpha) / (len(rows_c) + alpha *
len(values[j]))
    return classes, priors, cond
```

```
def predict(x, model):
    classes, priors, cond = model
    scores = {}
    for c in classes:
        p = priors[c]
        for j, v in enumerate(x):
            p *= cond.get((c, j, v), 1e-9)
        scores[c] = p
    return max(scores, key=scores.get), scores

def confusion_matrix(actual, predicted, labels):
    matrix = [[0] * len(labels) for _ in labels]
    for a, p in zip(actual, predicted):
        matrix[labels.index(a)][labels.index(p)] += 1
    return matrix

def show_results(actual, predicted, labels):
    matrix = confusion_matrix(actual, predicted, labels)
    w = max(len(l) for l in labels) + 8
    print("\nConfusion Matrix (rows = actual, columns = predicted)")
    print(" " * (w + 2) + "".join(("Pred " + l).rjust(w) for l in labels))
    for label, row in zip(labels, matrix):
        print(("Actual " + label).ljust(w + 2) + "".join(str(v).rjust(w) for v in
row))
    correct = sum(matrix[i][i] for i in range(len(labels)))
    print("\nAccuracy = %d / %d = %.2f%%" % (correct, len(actual), correct /
len(actual) * 100))

# Training data: Income, Employment, CreditHistory -> LoanApproved
train = [
    ['High', 'Salaried', 'Good', 'Yes'], ['High', 'SelfEmployed', 'Good', 'Yes'],
    ['High', 'Salaried', 'Poor', 'No'], ['Medium', 'Salaried', 'Good', 'Yes'],
    ['Medium', 'SelfEmployed', 'Poor', 'No'], ['Medium', 'SelfEmployed', 'Good',
'Yes'],
    ['Low', 'Salaried', 'Good', 'Yes'], ['Low', 'SelfEmployed', 'Good', 'No'],
    ['Low', 'Salaried', 'Poor', 'No'], ['Low', 'SelfEmployed', 'Poor', 'No'],
    ['Medium', 'Salaried', 'Poor', 'No'], ['High', 'SelfEmployed', 'Poor', 'No']]
test = [
    ['High', 'Salaried', 'Good', 'Yes'], ['Low', 'SelfEmployed', 'Poor', 'No'],
    ['Medium', 'Salaried', 'Poor', 'No'], ['Low', 'Salaried', 'Good', 'Yes'],
    ['High', 'SelfEmployed', 'Good', 'Yes'], ['Medium', 'SelfEmployed', 'Poor',
'Yes']]

X_train, y_train = [r[:-1] for r in train], [r[-1] for r in train]
X_test, y_test = [r[:-1] for r in test], [r[-1] for r in test]

model = train_nb(X_train, y_train)
print("Prior probabilities:", {c: round(p, 4) for c, p in model[1].items()})

predicted = []
for x, actual in zip(X_test, y_test):
    label, scores = predict(x, model)
    predicted.append(label)
    print(x, " actual =", actual, " predicted =", label)

show_results(y_test, predicted, ["No", "Yes"])
```

8. Output

```
>>>

===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/6.2.py ====
Prior probabilities: {'No': 0.5833, 'Yes': 0.4167}
['High', 'Salaried', 'Good']  actual = Yes  predicted = Yes
['Low', 'SelfEmployed', 'Poor']  actual = No  predicted = No
['Medium', 'Salaried', 'Poor']  actual = No  predicted = No
['Low', 'Salaried', 'Good']  actual = Yes  predicted = Yes
['High', 'SelfEmployed', 'Good']  actual = Yes  predicted = Yes
['Medium', 'SelfEmployed', 'Poor']  actual = Yes  predicted = No

Confusion Matrix (rows = actual, columns = predicted)
                 Pred No   Pred Yes
Actual No              2          0
Actual Yes             1          3

Accuracy = 5 / 6 = 83.33%
>>>
```

9. Result / Conclusion

The Naïve Bayes classifier was implemented successfully. The confusion matrix was displayed and the accuracy on the test data is 83.33% (1 test sample(s) misclassified).

Subproblem 3: Student Result Prediction (Numeric Features, Gaussian)

1. Aim

To implement Gaussian Naïve Bayes for numeric features and evaluate it using a confusion matrix and accuracy.

2. Concept

The features are study hours and previous exam score. Each feature is assumed to follow a normal distribution within each class, so only a mean and variance per class and feature are stored.

3. Dataset Description

Training data (12 students):

Study hours/day	Previous score	Result
1	35	Fail
2	40	Fail
2	55	Fail
3	45	Fail
3	60	Fail
4	50	Fail
5	65	Pass
6	70	Pass
6	80	Pass
7	75	Pass
8	85	Pass
9	90	Pass

Test data (5 students):

Study hours/day	Previous score	Result
2	50	Fail
5	60	Pass
7	85	Pass
4	48	Fail
5	52	Pass

4. Algorithm

1. Separate the data into training and test sets.
2. Compute the prior probability of each class.
3. For each class and each numeric feature, compute the mean μ and variance σ^2 from the training data.
4. For a test sample, evaluate the Gaussian density of every feature for each class and multiply the densities with the prior.
5. Predict the class with the largest value.

6. Build the confusion matrix and compute the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Priors: P(Fail) = 0.50; P(Pass) = 0.50.
2. Class Fail: feature 1: $\mu = 2.50$, $\sigma^2 = 0.92$; feature 2: $\mu = 47.50$, $\sigma^2 = 72.92$.
3. Class Pass: feature 1: $\mu = 6.83$, $\sigma^2 = 1.81$; feature 2: $\mu = 77.50$, $\sigma^2 = 72.92$.
4. Take test sample 1: study hours = 2, previous score = 50 (actual Fail). Gaussian density $f(x) = (1/\sqrt{(2\pi\sigma^2)}) \cdot e^{(-(x-\mu)^2/(2\sigma^2))}$:
5. Fail: f(hours) = 0.363565; f(score) = 0.044759; score = 0.50 × 0.363565 × 0.044759 = 8.136e-03.
6. Pass: f(hours) = 0.000460; f(score) = 0.000261; score = 0.50 × 0.000460 × 0.000261 = 6.017e-08.
7. The larger score belongs to Fail, so sample 1 is predicted as Fail. The same calculation is repeated for every test sample.
8. Confusion matrix (rows = actual, columns = predicted): actual Fail: [2, 0]; actual Pass: [1, 2], with columns in the order Fail, Pass.
9. Accuracy = (2 + 2) / 5 = 4/5 = 80.00%.
10. Misclassified test sample(s): sample 5 (actual Pass, predicted Fail).

6. Formulae

- Mean: $\mu = (1/n) \Sigma x_i$; variance: $\sigma^2 = (1/n) \Sigma (x_i - \mu)^2$
- Gaussian density: $f(x \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \cdot \exp(-(x - \mu)^2 / (2\sigma^2))$
- Decision rule: predicted class = argmax over C of $P(C) \cdot \Pi f(x_i \mid C)$
- Confusion matrix: rows = actual class, columns = predicted class; the diagonal holds the correct predictions
- Accuracy = (TP + TN) / (TP + TN + FP + FN) = (sum of the diagonal) / (total test samples)

7. Python Program

```
import math

def confusion_matrix(actual, predicted, labels):
    matrix = [[0] * len(labels) for _ in labels]
    for a, p in zip(actual, predicted):
        matrix[labels.index(a)][labels.index(p)] += 1
    return matrix

def show_results(actual, predicted, labels):
    matrix = confusion_matrix(actual, predicted, labels)
    w = max(len(l) for l in labels) + 8
    print("\nConfusion Matrix (rows = actual, columns = predicted)")
    print(" " * (w + 2) + "".join(("Pred " + l).rjust(w) for l in labels))
    for label, row in zip(labels, matrix):
        print(("Actual " + label).ljust(w + 2) + "".join(str(v).rjust(w) for v in
row))
    correct = sum(matrix[i][i] for i in range(len(labels)))
    print("\nAccuracy = %d / %d = %.2f%%" % (correct, len(actual), correct /
len(actual) * 100))

def fit_gaussian(X, y):
    model = {}
```

```
    for c in sorted(set(y)):
        rows = [x for x, label in zip(X, y) if label == c]
        stats = []
        for col in zip(*rows):
            mean = sum(col) / len(col)
            var = sum((v - mean) ** 2 for v in col) / len(col) + 1e-9
            stats.append((mean, var))
        model[c] = (len(rows) / len(X), stats)
    return model

def gaussian(x, mean, var):
    return math.exp(-((x - mean) ** 2) / (2 * var)) / math.sqrt(2 * math.pi * var)

def predict(x, model):
    scores = {}
    for c, (prior, stats) in model.items():
        score = prior
        for v, (mean, var) in zip(x, stats):
            score *= gaussian(v, mean, var)
        scores[c] = score
    return max(scores, key=scores.get), scores

# [study hours per day, previous exam score] -> Pass / Fail
X_train = [[1, 35], [2, 40], [2, 55], [3, 45], [3, 60], [4, 50],
           [5, 65], [6, 70], [6, 80], [7, 75], [8, 85], [9, 90]]
y_train = ['Fail'] * 6 + ['Pass'] * 6
X_test = [[2, 50], [5, 60], [7, 85], [4, 48], [5, 52]]
y_test = ['Fail', 'Pass', 'Pass', 'Fail', 'Pass']

model = fit_gaussian(X_train, y_train)
for c, (prior, stats) in model.items():
    print(c, "prior = %.2f" % prior, " (mean, variance) per feature:",
          [(round(m, 2), round(v, 2)) for m, v in stats])

predicted = []
for x, actual in zip(X_test, y_test):
    label, _ = predict(x, model)
    predicted.append(label)
    print(x, " actual =", actual, " predicted =", label)

show_results(y_test, predicted, ["Fail", "Pass"])
```

8. Output

```
>>>
    ===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/6.3.py ====
    Fail prior = 0.50  (mean, variance) per feature: [(2.5, 0.92), (47.5, 72.92)]
    Pass prior = 0.50  (mean, variance) per feature: [(6.83, 1.81), (77.5, 72.92)]
    [2, 50]  actual = Fail  predicted = Fail
    [5, 60]  actual = Pass  predicted = Pass
    [7, 85]  actual = Pass  predicted = Pass
    [4, 48]  actual = Fail  predicted = Fail
    [5, 52]  actual = Pass  predicted = Fail

    Confusion Matrix (rows = actual, columns = predicted)
                      Pred Fail   Pred Pass
    Actual Fail               2           0
    Actual Pass               1           2

    Accuracy = 4 / 5 = 80.00%
>>>
```

9. Result / Conclusion

The Naïve Bayes classifier was implemented successfully. The confusion matrix was displayed and the accuracy on the test data is 80.00% (1 test sample(s) misclassified).

Subproblem 4: Iris Classification (Three Classes)

1. Aim

To apply Gaussian Naïve Bayes to the Iris dataset and display a 3 × 3 confusion matrix with the accuracy.

2. Concept

The four flower measurements are modelled with a Gaussian distribution for each species. The program works with log probabilities to avoid numerical underflow. The data is split 105 training / 45 test.

3. Dataset Description

The Iris dataset (built into scikit-learn) has 150 samples, 4 numeric features, and 3 species with 50 samples each. Six representative rows are shown below.

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica

4. Algorithm

1. Separate the data into training and test sets.
2. Compute the prior probability of each class.
3. For each class and each numeric feature, compute the mean μ and variance σ^2 from the training data.
4. For a test sample, evaluate the Gaussian density of every feature for each class and multiply the densities with the prior.
5. Predict the class with the largest value.
6. Build the confusion matrix and compute the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. The 150 samples are shuffled (seed 42) and split into 105 training and 45 test samples.
2. Priors: P(setosa) = 40/105 = 0.3810; P(versicolor) = 33/105 = 0.3143; P(virginica) = 32/105 = 0.3048.
3. setosa: mean = (5.00, 3.43, 1.47, 0.25); variance = (0.117, 0.122, 0.032, 0.012).
4. versicolor: mean = (5.89, 2.75, 4.26, 1.33); variance = (0.233, 0.117, 0.237, 0.047).
5. virginica: mean = (6.62, 2.96, 5.58, 2.03); variance = (0.410, 0.136, 0.346, 0.077).
6. Take test sample 1: (4.6, 3.4, 1.4, 0.3), actual species setosa. Log-posterior score = ln P(C) + Σ ln $f(x_i \mid C)$: setosa = 0.551; versicolor = −34.806; virginica = −52.182. The largest score is setosa.
7. Confusion matrix (rows = actual, columns = predicted): actual setosa: [10, 0, 0]; actual versicolor: [0, 16, 1]; actual virginica: [0, 1, 17], with columns in the order setosa, versicolor, virginica.

8. Accuracy = (10 + 16 + 17) / 45 = 43/45 = 95.56%.
9. Misclassified test sample(s): sample 31 (actual versicolor, predicted virginica); sample 42 (actual virginica, predicted versicolor).

6. Formulae

- Gaussian density: $f(x \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \cdot \exp(-(x - \mu)^2 / (2\sigma^2))$
- Log score: $\ln P(C) + \Sigma \ln f(x_i \mid C)$; the class with the largest log score is predicted
- Confusion matrix is 3 × 3: rows = actual species, columns = predicted species
- Accuracy = (TP + TN) / (TP + TN + FP + FN) = (sum of the diagonal) / (total test samples)

7. Python Program

```
import numpy as np
from sklearn.datasets import load_iris

def confusion_matrix(actual, predicted, labels):
    matrix = [[0] * len(labels) for _ in labels]
    for a, p in zip(actual, predicted):
        matrix[labels.index(a)][labels.index(p)] += 1
    return matrix

def show_results(actual, predicted, labels):
    matrix = confusion_matrix(actual, predicted, labels)
    w = max(len(l) for l in labels) + 8
    print("\nConfusion Matrix (rows = actual, columns = predicted)")
    print(" " * (w + 2) + "".join(("Pred " + l).rjust(w) for l in labels))
    for label, row in zip(labels, matrix):
        print(("Actual " + label).ljust(w + 2) + "".join(str(v).rjust(w) for v in
row))
    correct = sum(matrix[i][i] for i in range(len(labels)))
    print("\nAccuracy = %d / %d = %.2f%%" % (correct, len(actual), correct /
len(actual) * 100))

iris = load_iris()
X, y = iris.data, iris.target
np.random.seed(42)
idx = np.random.permutation(len(X))
train_idx, test_idx = idx[:105], idx[105:]               # 70% train, 30% test
X_train, y_train, X_test, y_test = X[train_idx], y[train_idx], X[test_idx],
y[test_idx]

classes = np.unique(y_train)
priors = np.array([np.mean(y_train == c) for c in classes])
means = np.array([X_train[y_train == c].mean(axis=0) for c in classes])
vars_ = np.array([X_train[y_train == c].var(axis=0) + 1e-9 for c in classes])

def predict(x):
    log_post = np.log(priors) - 0.5 * np.sum(np.log(2 * np.pi * vars_) + (x - means)
** 2 / vars_, axis=1)
    return classes[np.argmax(log_post)]

predicted = [predict(x) for x in X_test]
names = list(iris.target_names)
show_results([names[a] for a in y_test], [names[p] for p in predicted], names)
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/6.4.py ====

Confusion Matrix (rows = actual, columns = predicted)
                          Pred setosa   Pred versicolor    Pred virginica
Actual setosa                      10                 0                 0
Actual versicolor                   0                16                 1
Actual virginica                    0                 1                17

Accuracy = 43 / 45 = 95.56%
>>>
```

9. Result / Conclusion

The Naïve Bayes classifier was implemented successfully. The confusion matrix was displayed and the accuracy on the test data is 95.56% (2 test sample(s) misclassified).

Subproblem 5: SMS Spam Detection (Text, Multinomial)

1. Aim

To implement a Multinomial Naïve Bayes text classifier that labels short messages as spam or ham and evaluate it with a confusion matrix and accuracy.

2. Concept

Each message is represented by its words. Word counts per class, with Laplace smoothing, give P(word | class). Log probabilities are added instead of multiplying small numbers.

3. Dataset Description

Training messages (8):

Message	Class
win money now	spam
free prize win now	spam
claim free money prize	spam
urgent win cash prize	spam
meeting at noon	ham
project report due today	ham
lunch with team today	ham
see you at the meeting	ham

Test messages (7):

Message	Class
free cash now	spam
team meeting today	ham
win a free prize	spam
report due at noon	ham
money prize urgent	spam
lunch at noon	ham
free lunch today	ham

4. Algorithm

1. Split the messages into training and test sets and build the vocabulary from the training messages.
2. Compute the prior probability of each class (ham, spam).
3. Count how often each word appears in each class and compute P(word | class) with Laplace smoothing.
4. For a test message, add log P(class) and the log probabilities of its known words for each class.
5. Predict the class with the highest score.
6. Build the confusion matrix and compute the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Vocabulary built from the training messages: V = 21 distinct words.
2. Priors: P(ham) = 4/8 = 0.50; P(spam) = 4/8 = 0.50. Total words: ham = 16, spam = 15.
3. Take test message 1: "free cash now" (actual spam). P(word | class) = (count + 1) / (total words in class + V):
4. ham: P(free | ham) = (0 + 1)/(16 + 21) = 0.0270; P(cash | ham) = (0 + 1)/(16 + 21) = 0.0270; P(now | ham) = (0 + 1)/(16 + 21) = 0.0270.
5. spam: P(free | spam) = (2 + 1)/(15 + 21) = 0.0833; P(cash | spam) = (1 + 1)/(15 + 21) = 0.0556; P(now | spam) = (2 + 1)/(15 + 21) = 0.0833.
6. Log scores = ln P(class) + Σ ln P(word | class): ham = −11.5259; spam = −8.5533. The larger score is spam, so message 1 is predicted as spam.
7. Confusion matrix (rows = actual, columns = predicted): actual ham: [4, 0]; actual spam: [0, 3], with columns in the order ham, spam.
8. Accuracy = (4 + 3) / 7 = 7/7 = 100.00%.
9. Every test sample is classified correctly.

6. Formulae

- Prior: P(C) = (messages of class C) / (all messages)
- Word likelihood with Laplace smoothing: P(w | C) = (count(w, C) + 1) / (total words in C + V)
- Score: ln P(C) + Σ ln P(w_i | C) over the known words of the message; the largest score wins
- Confusion matrix: rows = actual class, columns = predicted class; the diagonal holds the correct predictions
- Accuracy = (TP + TN) / (TP + TN + FP + FN) = (sum of the diagonal) / (total test samples)

7. Python Program

```
import math
from collections import Counter

def confusion_matrix(actual, predicted, labels):
    matrix = [[0] * len(labels) for _ in labels]
    for a, p in zip(actual, predicted):
        matrix[labels.index(a)][labels.index(p)] += 1
    return matrix

def show_results(actual, predicted, labels):
    matrix = confusion_matrix(actual, predicted, labels)
    w = max(len(l) for l in labels) + 8
    print("\nConfusion Matrix (rows = actual, columns = predicted)")
    print(" " * (w + 2) + "".join(("Pred " + l).rjust(w) for l in labels))
    for label, row in zip(labels, matrix):
        print(("Actual " + label).ljust(w + 2) + "".join(str(v).rjust(w) for v in
row))
    correct = sum(matrix[i][i] for i in range(len(labels)))
    print("\nAccuracy = %d / %d = %.2f%%" % (correct, len(actual), correct /
len(actual) * 100))

train = [("win money now", "spam"), ("free prize win now", "spam"),
         ("claim free money prize", "spam"), ("urgent win cash prize", "spam"),
         ("meeting at noon", "ham"), ("project report due today", "ham"),
         ("lunch with team today", "ham"), ("see you at the meeting", "ham")]
```

```
test = [("free cash now", "spam"), ("team meeting today", "ham"),
        ("win a free prize", "spam"), ("report due at noon", "ham"),
        ("money prize urgent", "spam"), ("lunch at noon", "ham"), ("free lunch today",
"ham")]

vocab = sorted(set(w for text, _ in train for w in text.split()))
classes = ["ham", "spam"]
priors = {c: sum(1 for _, l in train if l == c) / len(train) for c in classes}
word_counts = {c: Counter(w for text, l in train if l == c for w in text.split()) for
c in classes}
total_words = {c: sum(word_counts[c].values()) for c in classes}

def predict(text):
    scores = {}
    for c in classes:
        score = math.log(priors[c])
        for w in text.split():
            if w in vocab:                                    # ignore unseen words
                score += math.log((word_counts[c][w] + 1) / (total_words[c] +
len(vocab)))
        scores[c] = score
    return max(scores, key=scores.get), scores

print("Vocabulary size:", len(vocab))
predicted = []
for text, actual in test:
    label, scores = predict(text)
    predicted.append(label)
    print("%-22s actual = %-4s predicted = %s" % (text, actual, label))

show_results([l for _, l in test], predicted, classes)
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/6.5.py ====
Vocabulary size: 21
free cash now          actual = spam predicted = spam
team meeting today     actual = ham  predicted = ham
win a free prize       actual = spam predicted = spam
report due at noon     actual = ham  predicted = ham
money prize urgent     actual = spam predicted = spam
lunch at noon          actual = ham  predicted = ham
free lunch today       actual = ham  predicted = ham

Confusion Matrix (rows = actual, columns = predicted)
                  Pred ham   Pred spam
Actual ham               4           0
Actual spam              0           3

Accuracy = 7 / 7 = 100.00%
```

9. Result / Conclusion

The Naïve Bayes classifier was implemented successfully. The confusion matrix was displayed and the accuracy on the test data is 100.00% (no misclassified samples).

Overall Result of Experiment 6

The Naïve Bayes classifier was implemented from scratch for categorical, numeric (Gaussian), and text data. The accuracy shown in each program's confusion matrix is summarised below.

Subproblem	Application	Test accuracy	Misclassified samples
1	Play Tennis Prediction (Categorical Attributes)	85.71%	1
2	Loan Approval Prediction	83.33%	1
3	Student Result Prediction (Numeric Features, Gaussian)	80.00%	1
4	Iris Classification (Three Classes)	95.56%	2
5	SMS Spam Detection (Text, Multinomial)	100.00%	0

Final conclusion: Naïve Bayes gives fast and simple predictions even with the independence assumption, and the confusion matrix shows exactly which classes are confused with each other. Accuracy is the share of correct predictions, but on small test sets a single error changes it noticeably.

Notation: the outputs shown are the actual outputs of the programs listed in this record for the datasets given here.